## 2. Preparación y verificación del entorno

| Paso de la guía | Resultado |
|---|---|
| Frontend React + TypeScript + Vite | `gestion-academica-web/`. Proxy de Vite: `/api` → `http://localhost:9090`. Variable `VITE_API_BASE_URL=/api/v1` en `.env`. |
| Gestor de paquetes | Se incluye `pnpm-lock.yaml` (gestor indicado en la guía) y también `package-lock.json`. Funciona con `pnpm install` o con `npm install`. |
| API del grupo y puerto | `gestion-academica` corre en **9090** (`server.port=9090`), así que no hizo falta cambiar el proxy. |
| Verificación del contrato | Se revisaron los controladores y DTOs y se probaron todas las rutas con peticiones reales (colección en `postman/`). |
| Corrección de `httpClient.ts` | Los tipos y funciones quedaron a nivel de módulo (no anidados en una clase) y `env` se importa desde `src/shared/config/env.ts`. |
| Pantalla inicial | La pantalla "Hola" se reemplazó por el login y la vista de estudiantes. |

### Errores encontrados y corregidos en la API

Al probar la API con datos reales aparecieron dos errores que impedían consumirla (1 y 2) y un mensaje incorrecto (3):

1. **Ningún usuario podía consultar nada (403 incluso para ADMIN).** En `SecurityConfig.jwtAuthenticationConverter()` se usaba `setAuthoritiesClaimDelimiter("roles")` cuando lo correcto es `setAuthoritiesClaimName("roles")`. Spring buscaba los roles en el claim por defecto (`scope`), no encontraba ninguno y `@PreAuthorize` rechazaba todas las peticiones.
2. **`GET /api/v1/carreras` no podía autorizarse.** En `CarreraController.getAll()` decía `hasAnyRoles(...)`, una función que no existe en Spring Security. Se cambió por `hasAnyRole(...)`.
3. **Mensaje incorrecto en `JwtProperties`:** decía "al menos 256 bytes" cuando la validación exige 32 bytes (256 bits). Se corrigió el texto.

Observación (no se modificó): `GlobalExceptionHandler` importa `java.nio.file.AccessDeniedException` en lugar de la de Spring Security, así que ese handler nunca se ejecuta. Los 403 los responde correctamente `RestAccessDeniedHandler`.

## 3. Parte I – Contrato de la API

### Recurso elegido: Estudiantes

| Elemento | Valor verificado |
|---|---|
| Ruta GET (lista) | `GET /api/v1/estudiantes` |
| Ruta GET (detalle) | `GET /api/v1/estudiantes/{id}` |
| Parámetros | Lista: ninguno (la API no admite filtros ni paginación). Detalle: `id` (Long) en la ruta. |
| Autenticación | **Sí.** Header `Authorization: Bearer <accessToken>`, obtenido con `POST /api/v1/auth/login`. |

| Elemento | Valor verificado |
|---|---|
| Roles permitidos | `ADMIN` y `DOCENTE` |
| Sin token o token inválido | `401` → `{"status":401,"error":"UNAUTHORIZED","message":"Se requiere autenticación"}` o `"Token inválido o expirado"` |
| Rol sin permiso (ESTUDIANTE) | `403` → `{"status":403,"error":"Forbidden","message":"No tiene permisos para realizar esta operación"}` |
| Id inexistente | `404` → `{"error":"Not Found","message":"Estudiante con id 999 no encontrado","status":"404 NOT_FOUND"}` |

Respuesta real de `GET /api/v1/estudiantes` (200):

```
[
  { "id": 1, "nombre": "Ana López", "correo": "ana.lopez@uam.edu.ni", "edad": 20, "carreraId": 1 },
  { "id": 2, "nombre": "Carlos Pérez", "correo": "carlos.perez@uam.edu.ni", "edad": 22,
"carreraId": 1 }
]
```

Con la base vacía la respuesta es `200` con `[]`. Una lista vacía no es un error.

## Endpoints de apoyo

| Endpoint | Uso en el frontend |
|---|---|
| `POST /api/v1/auth/login` (público) | Body `{username, password}` → `{accessToken, tokenType:"Bearer", expiresIn:3600, username, rol}` |
| `GET /api/v1/carreras` (ADMIN, DOCENTE, ESTUDIANTE) | `[{id, codigo, nombre}]`, para mostrar el nombre de la carrera en lugar de `carreraId` |

## Campos mostrados y tipos TypeScript

| Campo JSON | Tipo Java | Tipo TypeScript | Se muestra como |
|---|---|---|---|
| `id` | Long | `number` | ID |
| `nombre` | String | `string` | Nombre |
| `correo` | String | `string` | Correo |
| `edad` | Integer | `number` | Edad |
| `carreraId` | Long | `number` | Nombre de la carrera (se cruza con `/carreras`) |

Observación: el cuerpo de error no es uniforme. `status` es un número en los errores de seguridad (`401`) y un texto en los de `GlobalExceptionHandler` (`"404 NOT_FOUND"`). Por eso `ApiErrorBody.status` es `number | string` y el frontend usa siempre el código HTTP de la respuesta, no ese campo.

## 4. Parte II – Integración con React

### Estructura

```
src/
├── app/                    App.tsx (sesión y layout) · queryClient.ts
├── shared/
│   ├── api/httpClient.ts   cliente HTTP compartido (URL base, Bearer, errores, 204)
│   ├── auth/tokenStorage.ts
│   ├── config/env.ts       lectura de VITE_API_BASE_URL
│   └── components/EstadoVista.tsx   carga / error / vacío
└── features/
    ├── auth/               types · services/authService · pages/LoginPage
    ├── carreras/           types · services · hooks/useCarreras
    └── estudiantes/        types · services/estudiantesService · hooks/useEstudiantes
                            components/EstudiantesTable · EstudianteDetalle · pages/EstudiantesPage
```

Flujo: **Componente → hook (TanStack Query) → servicio → httpClient → proxy de Vite → API**. Ningún componente llama a `fetch` directamente.

### Decisiones de implementación

- **TanStack Query en lugar de `useEffect` + `useState`.** Ya venía incluido en el proyecto y resuelve la caché, los estados `isPending` / `isError` / `isSuccess`, la cancelación (pasa el `signal` del `AbortController` a `fetch`) y el reintento. Las solicitudes nunca se hacen durante el render.
- **Reintentos selectivos.** Solo se reintenta una vez ante fallas de red o errores 5xx. Reintentar 401, 403 o 404 no cambia el resultado.
- **Cuatro estados visibles:**
  - **Carga:** spinner.
  - **Datos:** tabla.
  - **Vacío:** "No hay estudiantes registrados" (la consulta respondió 200 con `[]`).
  - **Error:** mensaje claro y botón *Reintentar*.
- **Tipos de error en `httpClient`:**
  - `http`: la API respondió con 4xx o 5xx; se usa el `message` del backend.
  - `network`: el backend no responde. Con el proxy de Vite esto llega como un 500 sin cuerpo JSON.
  - `parse`: el JSON no es válido.

  Ante un `204 No Content` no se llama a `response.json()`.
- **401:** el cliente HTTP avisa a `App`, que borra la sesión, limpia la caché y vuelve al login con el mensaje "Tu sesión expiró...".
- **403:** se muestra el mensaje de permisos del backend y se explica qué roles tienen acceso. No se ofrece *Reintentar* porque el resultado sería el mismo; ese botón solo aparece ante fallas de red o errores 5xx. Se puede comprobar con el usuario `estudiante1`.
- **Token:** se guarda en `sessionStorage` (se borra al cerrar la pestaña) y no en `localStorage`. Nunca se muestra en pantalla ni se registra en la consola.
- **Dato derivado:** el nombre de la carrera se calcula durante el render a partir de la lista de carreras; no se duplica en el estado.

### Actividad 3 – Opción A: detalle de un registro

Al hacer clic en una fila (o pulsar Enter sobre ella) se consulta `GET /api/v1/estudiantes/{id}` con `useQuery({ enabled: id !== null })` y se muestra un panel lateral con sus propios estados de carga y error. La opción B no se eligió porque la API no admite parámetros de filtro, y la guía pide no suponer endpoints.

## 5. Parte III – Pruebas funcionales

Pruebas ejecutadas el 04/11/2026 contra la API real (Spring Boot + PostgreSQL 18).

| # | Prueba | Esperado | Obtenido | Estado |
|---|---|---|---|---|
| 1 | `GET /estudiantes` sin token | 401 | 401 "Se requiere autenticación" | ✅ |
| 2 | Login con contraseña incorrecta | 401 | 401 "Usuario o contraseña incorrectos" | ✅ |
| 3 | Login `admin` | 200 + token | 200, `tokenType: Bearer`, `expiresIn: 3600`, `rol: ADMIN` | ✅ |
| 4 | `GET /estudiantes` como ADMIN (código original) | 200 | **403** (error de `SecurityConfig`) | ❌ → corregido |
| 5 | `GET /estudiantes` como ADMIN con la base vacía | 200 `[]` | 200 `[]` → estado vacío | ✅ |
| 6 | `GET /estudiantes` como ADMIN con datos | 200 + lista | 200, 3 registros | ✅ |
| 7 | `GET /estudiantes/2` | 200 | 200, Carlos Pérez | ✅ |
| 8 | `GET /estudiantes/999` | 404 | 404 "Estudiante con id 999 no encontrado" | ✅ |
| 9 | `GET /estudiantes` como DOCENTE | 200 | 200 | ✅ |
| 10 | `GET /estudiantes` como ESTUDIANTE | 403 | 403 "No tiene permisos..." | ✅ |
| 11 | `GET /carreras` como ESTUDIANTE | 200 | 200 | ✅ |
| 12 | Token con firma inválida | 401 | 401 "Token inválido o expirado" | ✅ |
| 13 | `GET /api/v1/estudiantes` a través del proxy de Vite (`:5173`) | 200 + mismos datos | 200, mismos datos que la petición directa a `:9090` | ✅ |
| 14 | Proxy con la API detenida | Error de conexión | El proxy responde 500 sin JSON → el frontend muestra "No se pudo conectar con la API" | ✅ |

### Pruebas de la interfaz en el navegador (Chrome)

| # | Escenario | Resultado observado | Estado |
|---|---|---|---|
| 15 | Login con contraseña incorrecta | "Usuario o contraseña incorrectos." bajo el formulario | ✅ |
| 16 | Login `docente1` | Tabla con 3 estudiantes y el nombre de su carrera | ✅ |
| 17 | Clic en una fila | Panel "Detalle del estudiante" con los datos de `GET /estudiantes/2` | ✅ |
| 18 | Botón **Actualizar** | Network: `GET http://localhost:5173/api/v1/estudiantes` → 200 | ✅ |

| # | Escenario | Resultado observado | Estado |
|---|---|---|---|
| 19 | Login `estudiante1` (rol sin permiso) | "No tiene permisos para realizar esta operación" + roles permitidos | ✅ |
| 20 | API detenida + **Actualizar** | El botón muestra "Actualizando...", luego "No se pudo conectar con la API..." y *Reintentar* | ✅ |
| 21 | Cerrar sesión | Se borra la sesión y vuelve al formulario de login | ✅ |

**Contraste en el navegador (DevTools → Network):** la URL que se ve es `http://localhost:5173/api/v1/estudiantes` (ruta relativa, mismo origen). El proxy la reenvía a `http://localhost:9090/api/v1/estudiantes`. El código HTTP y el JSON coinciden con los de Postman. En las capturas que se compartan hay que ocultar el header `Authorization`.

## 6. Análisis y reflexión

**1. ¿Qué función cumplen `VITE_API_BASE_URL` y el proxy de Vite?** `VITE_API_BASE_URL` concentra en un solo lugar el prefijo de la API (`/api/v1`). Para cambiar de entorno basta con editar `.env`, sin tocar componentes ni servicios. El proxy de Vite hace que, en desarrollo, el navegador llame a su mismo origen (`localhost:5173`) y Vite reenvíe la petición a Spring Boot (`localhost:9090`). Así no hay bloqueos de CORS y el código usa rutas relativas. Las variables `VITE_*` llegan al navegador, por eso nunca deben contener secretos.

**2. ¿Por qué conviene separar el acceso HTTP de los componentes React?** Porque cada pieza tiene una sola responsabilidad. La URL base, el header `Bearer`, la interpretación de 401/403/404/5xx y el caso 204 se escriben una sola vez en `httpClient`. Los servicios describen los endpoints y los componentes solo muestran datos. Si cambia el token, la URL o el manejo de errores, se modifica un archivo y no todas las pantallas. Además, los componentes son más fáciles de leer y de probar.

**3. ¿Qué diferencia hay entre manejar datos, carga y error mediante estado?** Son tres situaciones distintas que la interfaz debe representar de forma explícita y excluyente. *Carga* indica que la petición sigue en curso. *Datos* indica una respuesta correcta, que puede traer registros o una lista vacía (que no es un error). *Error* indica que la petición falló (red, 4xx o 5xx) y requiere un mensaje y una acción, como reintentar o volver a iniciar sesión. Modelarlos como estado evita pantallas en blanco y combinaciones imposibles, como mostrar "cargando" y un error al mismo tiempo.

**4. ¿Qué información del contrato de la API fue necesario verificar antes de construir el frontend?**

- El puerto (9090) y el prefijo (`/api/v1`).
- Las rutas y métodos reales.
- La forma exacta del JSON: `carreraId` es un número, no un objeto carrera.
- Que todas las rutas, excepto el login, exigen Bearer Token, y qué roles admite cada una.
- La respuesta del login (`accessToken`, `expiresIn`, `rol`).
- El formato de los errores 401, 403 y 404, que no es uniforme.
- Que no existen parámetros de búsqueda ni de paginación.

Gracias a esa verificación se detectó que la API, tal como estaba, respondía 403 a todos los usuarios.

**5. ¿Qué mejoras incorporaríamos para facilitar el mantenimiento?**

- Unificar el formato de error del backend (`status` siempre numérico) y documentar la API con OpenAPI/Swagger.
- Generar los tipos TypeScript desde ese contrato, o validar las respuestas con Zod.
- Agregar paginación y filtros en `GET /estudiantes`.

- Incluir en `EstudianteResponse` el nombre de la carrera, para evitar la segunda consulta.
- Pruebas automáticas: de integración en Spring ( `@WebMvcTest` con roles) y en React (Testing Library + MSW).
- Renovación del token (refresh token) y un archivo `.env.example` documentado.

---